

CS481 HW5

Tomas Salaz

May 6, 2026

Q1: Journaling

1. The ordering of `txb`, `i`, `b`, `Db` and `txe` is important when writing to the journal since it enforces consistency when doing the "write-ahead" to the journal. The most important are the `txb` and `txe` since they signal the beginning and end of the transaction. If this order is violated we can have inconsistent data and corrupt the file system and the recovery system.
2. `txe` is written last because it essentially acts as a "Commit" point for the journal to be written to the disk. If `txe` is not last, then the OS assumes the journal is correct so we call `fsync()` which pushes the journal to the disk. In the future, file recovery will be corrupt since the journal is wrong and there will not be a way to recover the files.

Q2: Check Pointing

1. There are 3 steps to verify that completeness. (1) Get the total file size using `lseek()` and check that size is equal to `txe.bytes`. (2) Use `lseek()` to jump `txe.bytes` to from the end of the files, storing that information into into a `buffer` using `read()` as we traverse back. (3) use `memcmp()` to compare the last bits in the `buffer` to the expected bytes before the `txe` block.

As long as the read returns the correct amount of bytes and the comparison passes, we open `data.txt` and copy the journal contents over then push it to disk.

2. If the system crashes between `Db` and `txe`, we have to discard everything in `journal.txt` and `data.txt` should not be updated. This is implemented in `checkpoint()` when we call `memcmp` to make sure the end of the `buffer` and the end of `journal.txt` is the same.

When the system comes back and recovers, we should rollback to the state before the last `txe` marker since we know that this is a completed commit to the disk. We loose the write but the data maintains correctness, then we can continue to do routines from there.